Web プログラミング

2017 年 3 月 24 日

五十嵐 浩人

目 次

1 HTML **5**
1.1 HTML5 ボイラープレートをデフォルトスタータテンプレートとして組み込む 5
1.2 フォーム 6
1.2.1 フォームで使用される要素 6
1.2.2 input 要素の type 属性 6
1.2.3 新しい属性 7
1.2.4 datalist 要素 7
1.2.5 ウィジェット 7
1.2.6 ユーザへの情報表示 7
1.2.7 クライアント側での値の検証 7
1.2.8 Constraint Validation API 7
1.2.9 フォームと CSS 7
1.3 HTML5 の新しい要素 9
1.4 WAI-ARIA 9
1.5 意味に基づくマークアップ 9
1.6 マイクロフォーマット 9
1.7 RDFa 9
1.8 マイクロデータ 9
1.9 データ属性 9
1.10 Web コンポーネント 9

2 CSS **9**
2.1 CSS とは 9
2.2 スタイルシートのインポート 10
2.3 埋め込みによるスタイルの設定 10
2.4 CSS ルール 10

2.5 スタイルのタイプ 10
2.6 CSS セレクタ 10
2.7 CSS のカスケード処理 10
2.8 との違い 10
2.9 計測単位 10
2.10 CSS のカラー 10
2.11 テキスト 10
2.11.1 テキストのフォント 10
2.11.2 テキストのスタイル 12
2.11.3 テキストエフェクト 13
2.12 ボックスモデル 14
2.12.1 ボックスの 4 つの線の指定 14
2.12.2 border-color プロパティ 15
2.12.3 border-width プロパティ 15
2.13 position プロパティ 15
2.13.1 絶対配置 15
2.13.2 相対配置 16
2.13.3 固定配置 16
2.14 マルチカラムレイアウト 17
2.15 Flexbox 17
2.15.1 Flexbox コンテナの設定 17
2.15.2 Flexbox アイテムの設定 17
2.16 擬似クラス 18
2.17 擬似要素 18
2.18 ルールの省略要素 18
2.19 CSS3 の属性セレクタ 18
2.20 box-sizing プロパティ 18
2.21 CSS3 の背景 18
2.21.1 テキストエフェクト 18
2.22 CSS3 の境界 18
2.23 ボックスシャドウ 18
2.24 要素のオーバーフロー 18
2.25 カラーと不透明度 18
2.26 変換 18

3 JavaScript 18
3.1 データ型と変数 18
3.2 演算子と文 18
3.2.1 JavaScript の文の形式 18
3.2.2 代入文 18

3.2.3 条件文とプログラムの流れ 18
3.2.4 条件式で使う演算子 19
3.2.5 論理演算子 19
3.2.6 繰り返し 19
3.3 JavaScript オブジェクト 20
3.3.1 単純データ型とオブジェクト 20
3.3.2 Boolean、Number、String 20
3.3.3 正規表現と RegExp 20
3.3.4 Date 20
3.3.5 Math 21
3.3.6 Array 21
3.4 プロトタイプパターンによるオブジェクト 21
3.4.1 ファクトリ関数での Object.create の使用 21
3.4.2 古いブラウザのための Object.create 21
3.4.3 オブジェクトコンストラクタにはファクトリパターンを使う 22
3.5 関数 24
3.5.1 関数と無名関数 24
3.5.2 自己実行型無名関数 25
3.5.3 関数リテラル 25
3.5.4 無名関数 26
3.5.5 関数の種類 26
3.5.6 関数のスコープ 26
3.6 トラブルへの対処、デバッグ、異種ブラウザ間の問題 26

4 Web API 26
4.1 イベント 26
4.1.1 イベントの監視 26
4.1.2 イベント処理のキャンセル - バブリングの中止 26
4.1.3 イベント処理のキャンセル - デフォルトのアクションの中止 27
4.1.4 イベント移譲 27
4.1.5 イベントの種類 27
4.1.6 イベントモデルの種類 28
4.1.7 イベントの発生順序 28
4.1.8 イベントオブジェクトのプロパティ 29
4.1.9 カスタムイベント 30
4.1.10 パブリッシュ/サブスクライブパターン 30
4.2 フォームと検証 30
4.3 BOM(ブラウザオブジェクトモデル) 30

4.3.1 BOM の構造 30
4.3.2 window オブジェクト 30
4.3.3 ウィンドウの作成と制御 32
4.3.4 navigator オブジェクト 34
4.3.5 location オブジェクト 35
4.3.6 タイマー 36
4.3.7 history オブジェクト 36
4.3.8 screen オブジェクト 37
4.4 クッキーとその後継技術 38
4.5 WebStorage API 38
4.5.1 API 38
4.6 DOM(ドキュメントオブジェクトモデル) 39
4.6.1 DOM の木構造 39
4.6.2 Node オブジェクトのプロパティとメソッド 39
4.6.3 木構造の変更 40
4.7 Ajax 41
4.7.1 XMLHttpRequst 42
4.7.2 XMLHttpRequst のメソッド 43
4.8 デバイス操作 API 44
4.9 画像とグラフィック 44

5 SPA クライアント 44
5.1 シェルの開発 44
5.2 機能モジュールの追加 44
5.3 モデルの構築 44
5.4 モデルとデータモジュールの完成 44

6 SPA サーバ 44
6.1 Web サーバ 45
6.1.1 サーバの役割 45
6.1.2 Node.js 45
6.1.3 request オブジェクト 45
6.1.4 高度なルーティング 46
6.1.5 express-session によるセッション管理 46
6.1.6 WebSocket と Socket.io 48
6.1.7 まとめ 51
6.2 サーバデータベース 51
6.2.1 MongDB シェルコマンド 51
6.3 SPA を本番環境に備える 53

7 SPA クライアント **53**
7.1 シェルの開発 53
7.2 機能モジュールの追加 53
7.3 モデルの構築 53
7.4 モデルとデータモジュールの完成 53
7.4.1 データバインディングと jQuery 53
7.4.2 オブザーバ 54

8 JavaScript のコーディングルール **54**
8.1 コードレイアウトとコメント 54
8.2 変数名 54
8.2.1 命名規則を使ってコメントを減らし改善する 54
8.2.2 命名規則を使う 54
8.2.3 関数の命名 54
8.3 変数の宣言と割り当て 55
8.4 関数 55
8.5 名前空間 55
8.6 ファイル名とレイアウト 55
8.7 構文 55
8.7.1 ラベル 55
8.7.2 文 55
8.8 コードを検証する 57
8.9 モジュール用のテンプレート 57

9 参考文献 **57**

1 HTML

1.1 HTML5 ボイラープレートをデフォルトスタータテンプレートとして組み込む

HTML5 ボイラープレートのホームページには「web のもっとも有名なフロントエンドテンプレート」と書かれています。

HTML5 BOILERPLATE ここから HTML5 Boilerplate をダウンロードして使用します。

1.2 フォーム

1.2.1 フォームで使用される要素

フォームで使用される要素のうちデータを入力する要素には次のようなものがあります。

- input 要素
- textarea 要素
- checkbox 要素
- radio 要素
- select 要素
- button 要素

ユーザに対して情報を提供する要素には次のようなものがあります。

- progress 要素
- meter 要素
- output 要素

1.2.2 input 要素の type 属性

次は type 属性にセットできる値です。

値	用途
hidden	不可視データ
text	改行を含まないテキスト
search	検索用キーワード
tel	電話番号
url	http://から始まる URL
email	メールアドレス
password	パスワード
datetime	年月日と時分秒
date	年月日
month	年月
week	年内の第何週か
time	時分秒
datetime-local	年月日と時分秒
number	数値
range	スライダー式の数値入力

値	用途
color	色の選択
checkbox	チェックボックス
radio	ラジオボタン
file	ファイルの選択
submit	フォーム内容の送信 (送信ボタン)
image	画像を使ったボタン
reset	フォーム内容の全消去 (リセットボタン)
button	汎用ボタン

1.2.3 新しい属性

1.2.4 datalist 要素

1.2.5 ウィジェット

1.2.6 ユーザへの情報表示

1.2.7 クライアント側での値の検証

1.2.8 Constraint Validation API

1.2.9 フォームと CSS

HTML5 では UI element states pseudo-class と呼ばれる CSS の擬似クラスが追加されています。

擬似クラス	適用される条件
:required	required 属性がセットされている
:optional	required 属性がセットされていない
:valid	検証結果が正当な値
:invalid	検証結果が不当な値
:disable	disable 属性がセットされている
:enable	enable 属性がセットされている
:read-only	readonly 属性に true がセットされている
:read-write	readonly 属性に true がセットされていない
:indeterminate	radio タイプと checkbox タイプでチェックも解除もしていない

擬似クラス	適用される条件
:checked	radio タイプと checkbox タイプでチェックした状態
なし	radio タイプと checkbox タイプでチェックが解除された状態
:in-range	number タイプと range タイプで指定可能な範囲であれば
:out-of-range	number タイプと range タイプで指定可能な範囲でなければ

使用例は次になります。

```
input:required { border-color: black; }
input:optional { border-color: gray; }
input:valid { color: green; }
input:invalid { color: red; }
```

radio タイプと checkbox タイプでチェックが解除された状態に対応する擬似クラスはないため次のように否定のセレクタを使います。

```
input[type='checkbox']:not(:checked) { ... }
```

1.3 HTML5 の新しい要素

1.4 WAI-ARIA

1.5 意味に基づくマークアップ

1.6 マイクロフォーマット

1.7 RDFa

1.8 マイクロデータ

1.9 データ属性

1.10 Web コンポーネント

2 CSS

2.1 CSS とは

CSS(Cascading Style Sheets) は、Web ページの背景、フォント、色などの「見た目」を変えるために使用されます。 CSS1 から CSS3 までの簡単な歴史を示します。

- CSS1

  1996 年、W3C(WWW コンソーシアム) 勧告として公表されました。 1999 年に改訂され、2001 年時点ですべてのプラウザでサポートされるようになりました。

- CSS2

  1998 年に公表されました。 CSS2 の策定はその後 CSS2.1 に引き継がれ、現在この CSS2.1 が Web 標準とみなされています。

- CSS3

  2000 年に CSS3 の仕様の策定が開始されました。

2.2　スタイルシートのインポート

2.3　埋め込みによるスタイルの設定

2.4　CSS ルール

2.5　スタイルのタイプ

2.6　CSS セレクタ

2.7　CSS のカスケード処理

2.8　との違い

2.9　計測単位

2.10　CSS のカラー

2.11　テキスト

2.11.1　テキストのフォント

フォントのプロパティには、family と style、size、weight の 4 つがあります。

2.11.1.1　font-family

font-family プロパティには使用するフォントを設定します。

2.11.1.2　Web フォントによる font-family の設定

CSS3 から、Web からフォントをロードして表示できるようになりました。これを Web フォントと呼びます。@font-face プロパティを使って font-family に設定する Web フォントの名称を宣言します。

```
@font-face {
  font-family:FontName;
  src:url('FontName.otf');
}
```

2.11.1.2.1　Google の Web フォント

Web フォントを使用する方法の 1 つは、Google のサーバーからロードする方法です。

Google の Web フォントサイト

フォントの使い方は、HTML の<link>タグを使って、使用するフォントをロードします。

```
<link href="http://fonts.googleapis.com/css?family=Lobster' />
```

CSS でロードしたフォントを次のように適用します。

```
h1 {
  font-family: 'Lobster', arial, serif;
}
```

2.11.1.3　font-style

font-style プロパティには使用するフォントのスタイルを設定します。設定できるスタイルは以下になります。

スタイル	説明	書式
normal	標準体	{ font-style:normal; }
italic	イタリック体	{ font-style:italic; }
oblique	斜体	{ font-style:oblique; }

```
.normal {
  font-style  : normal;
}
.italic {
  font-style  : italic;
}
.oblique {
  font-style  : oblique;
}
```

2.11.1.4　font-size

font-size プロパティの設定方法には固定と相対の 2 つがあります。

固定で設定する例は

```
p {
  font-size : 14pt;
}
```

相対で設定する例は

```
h1 {
  font-size : 240%;
}
h2 {
  font-size : 200%;
}
h3 {
  font-size : 160%;
}
h4 {
  font-size : 120%;
}
```

2.11.1.5 font-weight

font-weight プロパティは、フォントの重さを設定するときに使います。フォントの重さとはフォントの太さのことです。このプロパティには 100 から 900 までの 100 刻みの値などが設定できますが、通常使用されるのは normal と bold です。

```
.bold {
  font-weight : bold;
}
```

2.11.2 テキストのスタイル

2.11.2.1 装飾

2.11.2.2 間隔

2.11.2.3 整列

2.11.2.4 大文字、小文字変換

2.11.2.5　インデント

2.11.2.6　カラー

CSS のカラーを参照して下さい。

2.11.3　テキストエフェクト

CSS3 からはテキストに影を表示したりやテキストを重ねたり、単語を折り返したりなど、さまざまなエフェクト可能になりました。

2.11.3.1　text-shadow プロパティ

テキストの影を表示します。引数は以下の順に設定します。この例だと水平方向と垂直方向に 3 ピクセルだけ影を離し、4 ピクセルのぼかし量で、ダークグレーの影を表示します。

1. 水平方向へのオフセット
2. 垂直方向へのオフセット
3. ぼかし量
4. 使用するカラー

```
text-shadow : 3px 3px 4px #444;
```

2.11.3.2　text-overflow プロパティ

CSS の overflow プロパティに hidden を設定して使用するときには、text-overflow プロパティを使って、領域からはみ出て切り詰められたテキストがあることを示す省略記号 (...) を、切り取りの直前に置くことができます (ellipsis は省略記号という意味です)。

```
text-overflow : ellipsis;
```

これを動作させるには、次の 3 つの要件が必要になります。

1. 要素の overflow プロパティには overflow:hidden など、visible 以外の値を設定します。
2. 要素の white-space プロパティにはテキストに制約を与える nowrap を設定します。
3. 要素の幅は切り詰めるテキストの幅よりも小さくします。

2.11.3.3　word-wrap プロパティ

テキストを切り詰める方法には、word-wrap プロパティに break-word 値を指定して長い行を折り返すという方法もあります。

```
word-swrap : break-word;
```

2.12　ボックスモデル

HTML 要素はすべて長方形のボックスの中に含まれると想定されます。 HTML 要素のまわりは「パディング」(詰め物)、「ボーダー」(境界領域)、「マージン」(余白領域) の 3 つのボックスが存在します。

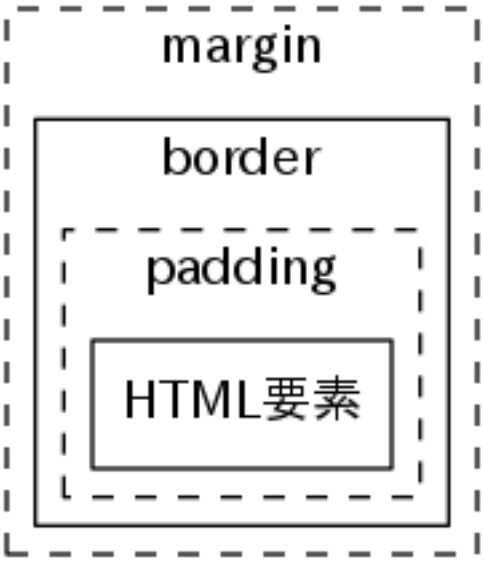

図 1: ボックス

2.12.1　ボックスの 4 つの線の指定

border-color プロパティ、border-width プロパティなどは上、下、右、左の 4 つの線を別々に設定できます。設定する値の数によりどの線の設定かを決めることができます。

2.12.2 border-color プロパティ

デフォルト値は HTML 要素の color プロパティの値になります。

値の数	設定される線
1	上、下、右、左の全てに同じ値が設定される
2	1 番目は上と下の線。 2 番目は左と右の線。
3	1 番目は上の線。 2 番目は左と右の線。 3 番目は下の線。
4	1 番目は上の線。 2 番目は右の線。 3 番目は左の線。 4 番目は下の線。

2.12.3 border-width プロパティ

1 つから 4 つまでの値を指定することが出来ます。数値を使って指定することもできますし、「thin」(細い)「medium」(中間)「thick」(太い) を指定することもできます。デフォルト値は「medium」です。

2.13 position プロパティ

Web ページ内の要素はドキュメントに記述された順番で配置されますが、要素の position プロパティをデフォルトの static(静的) から absolute(絶対) や relative(相対) または fixed(固定) に変更することで、その位置を変えることができます。

position プロパティに指定できる値の一覧

値	説明
static(デフォルト)	静的配置
absolute	絶対配置
relative	相対配置
fixed	固定配置

2.13.1 絶対配置

絶対配置に設定した要素はドキュメントから削除され、空いたスペースにはそのほかの要素が流し込まれます。絶対配置の要素はその後、top や right、

bottom、left プロパティを使ってドキュメントの好きな位置に持ってくることができます。この要素は他の要素の手前 (または奥) に来ます。

たとえば object という ID のオブジェクトを、ドキュメントの上端から下に 100 ピクセル、左端から 200 ピクセルの絶対位置に移動するには、オブジェクトに次のルールを適用します。

```
/* ブラウザの表示領域の左上隅を基点に、下に 100px、右に 200px 進んだ位置に配置 */
#object {
  position  : absolute;
  top       : 100px;
  left      : 200px;
}
```

2.13.2 相対配置

オブジェクトの本来の位置を基準に位置を指定する配置方法です。

たとえば object という ID のオブジェクトを、本来の位置から下に 10 ピクセル、左端から 10 ピクセル動かすには、オブジェクトに次のルールを適用します。

```
/* ブラウザの表示領域の左上隅を基点に、下に 100px、右に 200px 進んだ位置に配置 */
#object {
  position  : relative;
  top       : 10px;
  left      : 10px;
}
```

2.13.3 固定配置

このプロパティを適用されたオブジェクトは絶対位置に移動します。ドキュメントをスクロールしても位置は変わりません。

```
/* ブラウザの表示領域の左上隅を基点に、下に 100px、右に 200px 進んだ位置に配置 */
#object {
  position  : fixed;
  top       : 0px;
  left      : 0px;
}
```

2.14 マルチカラムレイアウト

2.15 Flexbox

Flexbox(Flexible Box) は UI 上にボタンあるいはメニューなどのフォーム要素やインタラクティブな領域が多数含まれる場合に使われます。

2.15.1 Flexbox コンテナの設定

Flexbox を利用するには、まず Flex コンテナ (flex container) を作成します。この要素の中に、すべての項目 (flex items) が子要素として配置されます。以下のように display 属性を指定するだけで、Flex コンテナは宣言できます。

```
E {
  display : flex;
}
```

デフォルトでは子要素は同じ幅で左右に並びます。

2.15.2 Flexbox アイテムの設定

プロパティ	説明
flex-grow	空白ができたときの子要素間の比率
flex-shrink	空白が足りないときの子要素間の比率
flex-basis	width プロパティと同等

2.16　擬似クラス

2.17　擬似要素

2.18　ルールの省略要素

2.19　CSS3 の属性セレクタ

2.20　box-sizing プロパティ

2.21　CSS3 の背景

2.21.1　テキストエフェクト

2.22　CSS3 の境界

2.23　ボックスシャドウ

2.24　要素のオーバーフロー

2.25　カラーと不透明度

2.26　変換

3　JavaScript

3.1　データ型と変数

3.2　演算子と文

3.2.1　JavaScript の文の形式

3.2.2　代入文

3.2.3　条件文とプログラムの流れ

3.2.3.1　if ... else

3.2.3.2 switch 文

switch 文は、条件式のとり得る値が多いときに便利です。式の計算結果により、ひとつまたは複数の選択肢を処理します。次のような構文を持ちます。

```
switch ( 式 ) {
  case ラベル 1:
    コードブロック;
    break;
  case ラベル 2:
    コードブロック;
    break;
  ...
  case ラベル n:
    コードブロック;
    break;
  default:
    コードブロック;
}
```

3.2.4 条件式で使う演算子

3.2.5 論理演算子

3.2.6 繰り返し

3.2.6.1 while ループ

3.2.6.2 do ... while ループ

3.2.6.3 for ループ

for ループは 2 種類あります。 for ループと for-in ループです。配列には for ループを使い、オブジェクトには for-in ループを使います。

for ループは次の形式です。 ~ for (初期値の設定; 条件の判定; 更新) { ... } ~

for-in ループは次の形式です。 ~ for (変数名 in オブジェクト) { ... } ~

3.3 JavaScript オブジェクト

3.3.1 単純データ型とオブジェクト

3.3.2 Boolean、Number、String

3.3.3 正規表現と RegExp

3.3.4 Date

Date オブジェクトは日時を扱うためのもので、このインスタンスを生成すると年、月、日、秒など、日時関連の情報を利用できます。

Date オブジェクトのメソッド:

メソッド	説明
getFullYear	「2010」など西暦の年 (4 桁) を返す
getMonth	月を 0(1 月) から 11(12 月) の数字で返す
getDate	日付を 1 から 31 の数字で返す
getDay	曜日を 0(日曜) から 6(土曜) の数字で返す
getHours	時を返す
getMinutes	分を返す
getSeconds	秒を返す
getMilleseconds	ミリ秒を返す
getTimezoneOffset	ローカル時刻と UTC の差分を分を単位として返します

これらの get メソッドには対応する set メソッドがあります。 set メソッドは Date 内の要素値を設定します。ただし、曜日を返す getDay に対応する set メソッドはありません。

日付の内部的な表現を出力するためのメソッドもあります。

メソッド	説明
toString	現地時刻を表す文字列 (英語) に変換
toGMTString	グリニッジ標準時を用いて文字列をフォーマット
toLocaleDateString	現在のロケールをフォーマット用いて日付を変換
toLocaleTimeString	現在のロケールをフォーマット用いて時刻を変換
toLocaleString	現在のロケールを用いて文字列を変換
toUTCString	UTC で文字列をフォーマット

```
var now = new Date();

now.toLocaleString();
// 2017/5/21 21:42:55 が返ります
```

3.3.5 Math

3.3.6 Array

3.4 プロトタイプパターンによるオブジェクト

プロトタイプパターンとは既存オブジェクトのコピーであるテンプレートをもとにオブジェクトを生成することです。

3.4.1 ファクトリ関数での Object.create の使用

```
var proto {
  sentence  : 4,
  probation : 2
};

var makePrisoner = function ( name, id ) {
  var prisoner = Object.create( proto );
  prisoner.name = name;
  prisoner.id = id;

  return prisoner;
};

var firstPrisoner = makePrisoner( 'Joe', '12A' );

var secondPrisoner = makePrisoner( 'Sam', '2BC' );
```

3.4.2 古いブラウザのための Object.create

Object.create は IE 9 以降、FIrefox 4 以降、Safari 5 以降、Chrome 5 以降で動作します。古いブラウザで互換性を保つためには、Object.create が存

在しないときには Object.create を定義し、すでに実装済みのブラウザでは Object.create をそのままにする必要があります。

```
// Object.create() をサポートするための特定のブラウザに依存しない方法

var objectCreate = function ( arg ) {
  if ( ! arg ) {
    return {};
  }
  function obj() {};
  obj.prototype = arg;
  return new obj;
};

Object.create = Object.create || objectCreate;
```

3.4.3 オブジェクトコンストラクタにはファクトリパターンを使う

哺乳類 (Mammal) コンストラクタを使って猫 (Cat) プロトタイプを作成し、猫 (Cat) プロトタイプを使って猫 (Cat) コンストラクタを生成する例を示します。

```
var createObject, extendObject,
  sayHello, sayText, makeMammal,
  catPrototype, makeCat, garfieldCat;

// 継承を設定するユーティリティ関数
// Object.create() を継承するための特定のブラウザに依存しないメソッド
// 新しい js エンジン (v1.8.5+) はネイティブにサポートする
var objectCreate = function ( arg ) {
  if ( ! arg ) {
    return {};
  }
  function obj() {};
  obj.prototype = arg;
  return new obj;
};

Object.create = Object.create || objectCreate;
```

```
// オブジェクトを拡張するためのユーティリティ関数
extendObject = function ( orig_obj, ext_obj ) {
  var key_name;

  for ( key_name in ext_obj ) {
    if ( ext_obj.hasOwnProperty( key_name ) ) {
      orig_obj[ key_name ] = ext_obj[ key_name ];
    }
  }
};

// オブジェクトメソッド
sayHello = function () {
  console.log( this.hello_text + ' says ' + this.name );
};

sayText = function ( text ) {
  console.warn( this.name + ' says ' + text );
};

// makeMammal コンストラクタ
makeMammal = function( arg_map ) {
  var mammal = {
    is_warn_blooded : true,
    has_fur         : true,
    leg_count       : 4,
    has_live_birth  : true,
    hello_text      : 'grunt',
    name            : 'anonymous',
    say_hello       : sayHello,
    say_text        : sayText
  };
  extendObject( mammal, arg_map );

  return mammal;

};

// makeMammal コンストラクタを使って cat プロトタイプを作成する
```

```
catPrototype = makeMammal({
  has_whiskers  : true,
  hello_text    : 'meow'
});

// cat コンストラクタ
makeCat = function ( arg_map ) {
  var cat = Object.create( catPrototype );
  extendObject( cat, arg_map );

  return cat;

};

// cat インスタンス
garfieldCat = makeCat({
  name        : 'Garfield',
  weight_lbs  : 8.6
});

// cat インスタンス呼び出し
garfieldCat.say_hello();
garfieldCat.say_text( 'Purr...' );
```

3.5 関数

JavaScript では関数は第一級オブジェクトです。

関数は変数や所定の属性に格納でき、関数呼び出しの引数として渡すこともできます。

3.5.1 関数と無名関数

関数は function というキーワードから始まり、関数名が続き、括弧に囲まれた引数 (なくてもよい)、そして関数本体が続きます。 JavaScript の関数の宣言は以下のようになります。

```
function foo () {}
```

変数に関数を格納する場合は以下のようになります。

```
var foo = function foo () {};
```

ローカル変数に無名関数を格納する場合は以下のようになります。

```
var foo = function () {};
```

関数は以下のように呼び出します。

```
foo();
```

3.5.2 自己実行型無名関数

関数を即座に実行する場合は自己実行型無名関数を使います。以下のようになります。

```
(function () {})();
```

自己実行型無名関数はグローバル名前空間に変数を追加しません。プライベート変数やプライベートメソッドを定義するときに使います。

3.5.2.1 関数の命名規則

3.5.2.2 宣言型の戻り値と引数

3.5.3 関数リテラル

3.5.3.1 再帰関数

3.5.3.2 クロージャ

3.5.3.3 コールバック関数

3.5.4 無名関数

3.5.5 関数の種類

3.5.6 関数のスコープ

3.6 トラブルへの対処、デバッグ、異種ブラウザ間の問題

4 Web API

4.1 イベント

4.1.1 イベントの監視

イベントを監視するには addEventListener() という関数を使います。この関数は type(イベントの種類。'click' など)、listener(イベントリスナの関数名)、useCapture の 3 つの引数を受け取ります。

useCapture が false もしくは何も指定されていない場合はバブリングの際にイベントリスナが呼び出されます。バブリングとはイベントが発生した HTML 要素から外側の HTML 要素の順にイベントリスナが呼び出される方式のことです。 useCapture が true の場合はキャプチャリングの際に呼び出されます。実際にはバブリングが使われることがほとんどです。

```
object.addEventListener( type, listener, useCapture );
```

イベントリスナを削除するには removeEventListener() を使います。 addEventListener で指定したのと同じ引数を指定する必要があります。

4.1.2 イベント処理のキャンセル - バブリングの中止

バブリングを中止するにはイベントオブジェクトの stopPropagation() を使います。この関数が呼び出されると、外側の要素で指定されているすべてのイベントリスナは呼び出されなくなります。 propagation はイベントの伝播と呼ばれることもあります。次の例の e は event です。

```
if ( typeof e.stopPropagation === 'function' ) {
  e.stopPropagation();
}
if ( typeof e.cancelBubble !== 'undefined' ) {
```

```
  e.cancelBubble = true;
}
```

4.1.3 イベント処理のキャンセル - デフォルトのアクションの中止

ブラウザにはリンクがクリックされると、新しいページを読み込むというデフォルトのアクションが定義されています。このデフォルトのアクションを中止するには 2 通りの方法があります。

- イベントオブジェクトの preventDefault() 関数を使う
- イベントリスナの返り値で false を返す

次の例の e は event です。

```
if ( typeof e.preventDefault === 'function' ) {
  e.preventDefault();
}
if ( typeof e.returnValue !== 'undefined' ) {
  e.returnValue = false;
}
```

4.1.4 イベント移譲

4.1.5 イベントの種類

ブラウザごとにサポートされているイベントの種類は異なります。
Quirksmode にイベントの一覧があります。

イベント	説明	関連オブジェクト
blur	オブジェクトがフォーカスを失ったとき	ウィンドウ要素とフォーム要素
focus	オブジェクトがフォーカスを取得したとき	ウィンドウ要素とフォーム要素
change	input 要素の入力内容が変更されたとき	フォーム要素の入力フィールド
click,	マウスでクリックしたとき	ほとんどのページ要素
doubleclick(dblclick)	マウスでダブルクリックしたとき	ほとんどのページ要素
keydown	キーを押したとき	ウェブページのドキュメント要素と一部
keyup	キーを離したとき	ウェブページのドキュメント要素と一部
keypress	キーを押して離したとき	ウェブページのドキュメント要素と一部
mousedown	マウスのボタンを押したとき	ほとんどのページ要素
mouseup	マウスのボタンを離したとき	ほとんどのページ要素

イベント	説明	関連オブジェクト
mouseover	マウスを要素に合わせたとき	ほとんどのページ要素
mouseout	要素からマウスを外したとき	ほとんどのページ要素
mousemove	マウスを移動したとき	ほとんどのページ要素
submit	フォームが提出されたとき	フォーム

イベントを処理するのはイベントリスナの役目です。イベントリスナは、イベントの名称に「on」という接頭辞を付けて表します。たとえば onload(ロードしたとき)、onblur(フォーカスを失ったとき)、onsubmit(提出したとき) といった具合です。

4.1.6 イベントモデルの種類

イベントの扱い方 (イベントモデル) は、大きく次の 3 種類に分けられます。このうちオリジナルイベントモデルはすべての JavaScript 対応ブラウザがサポートしていますが、残りふたつはブラウザによってサポートしている場合とサポートされていない場合があります。

- 「オリジナルイベントモデル」あるいは「DOM レベル 0」と呼ばれるもの。 HTML4 の標準で、ある程度規定されたもので、機能はやや限定されますが、JavaScript をサポートするすべてのブラウザで利用できるものです。初期の DOM の実装に基づくモデルです。
- 「標準イベントモデル」あるいは「DOM レベル 2」と呼ばれるもの。こちらは「DOM レベル 2」として標準化されたもので、オリジナルイベントモデルよりも強力な機能を提供します。 IE を除く最近のほとんどのブラウザサポートされています。
- Internet Explore のイベントモデル。残念ながら、IE は標準イベントモデルを (部分的にしか) サポートしていません。このため、利用したい機能によっては、ブラウザごとに異なる対応をしなければなりません。

4.1.7 イベントの発生順序

イベントの発生順序には 2 つの種類があります。

- イベントキャプチャリング (event capturing)

  外側の HTML 要素からイベントが発生した HTML 要素へと順にイベントリスナが呼び出されていきます。

- イベントバブリング (event bubbling)

  イベントが発生した要素から外側の HTML 要素へと順にイベントリスナが呼び出されていきます。

4.1.8 イベントオブジェクトのプロパティ

イベントの種類:

プロパティ	説明
type	イベントの種類
bubbles	イベントがバブリング中かどうかを表す真偽値

イベントが発生した環境についてのプロパティ:

プロパティ	説明
button	マウスボタンが押された際に、どのボタンかを表す
ctrlKey	Control キーがイベント発生時に押されたかを示す論理値
altKey	Alt キーがイベント発生時に押されたかを示す論理値
shiftKey	Shift キーがイベント発生時に押されたかを示す論理値
metaKey	Meta キーがイベント発生時に押されたかを示す論理値

キーボードイベントに固有のプロパティ:

プロパティ	説明
isChar	イベントに文字が含まれているかどうかを表す真偽値
charCode	押された文字に対応する Unicode 値 (keypress イベントのみ)
keyCode	文字ではないキーの Unicode 値
which	(文字化とうかにかかわらず) 押されたキーの Unicode 値

イベントが発生した場所:

プロパティ	説明
PageX	ページ (ビューポート) 内での相対的な座標の X 値
PageY	ページ (ビューポート) 内での相対的な座標の Y 値
screenX	イベントが発生したコンピュータの画面の X 座標

プロパティ	説明
screenY	イベントが発生したコンピュータの画面の Y 座標
clientX	イベントが発生したクライアント領域 (ウィンドウ内) の X 座標
clientY	イベントが発生したクライアント領域 (ウィンドウ内) の Y 座標

イベントに関連づけられている要素:

プロパティ	説明
currentTarget	バブリング中での現在の DOM 要素
target	オリジナルの DOM 要素
originalTarget	オリジナルの DOM 要素
relatedTarget	イベントに関連する別の要素 (イベントの種類による)

4.1.9 カスタムイベント

4.1.10 パブリッシュ/サブスクライブパターン

4.2 フォームと検証

4.3 BOM(ブラウザオブジェクトモデル)

Web ブラウザには BOM(Brower Object Model: ブラウザオブジェクトモデル) と呼ばれるオブジェクトの集合が用意されています。

4.3.1 BOM の構造

BOM のオブジェクトは下図に示すように階層構造になっています。

4.3.2 window オブジェクト

window オブジェクトはウェブページの内容、その周囲を取り巻く部分、そのページ内のイベントなど、ブラウザ全体の環境を包含したものとなっています。

window オブジェクトのプロパティとメソッドは次の 4 種類に分類されます。

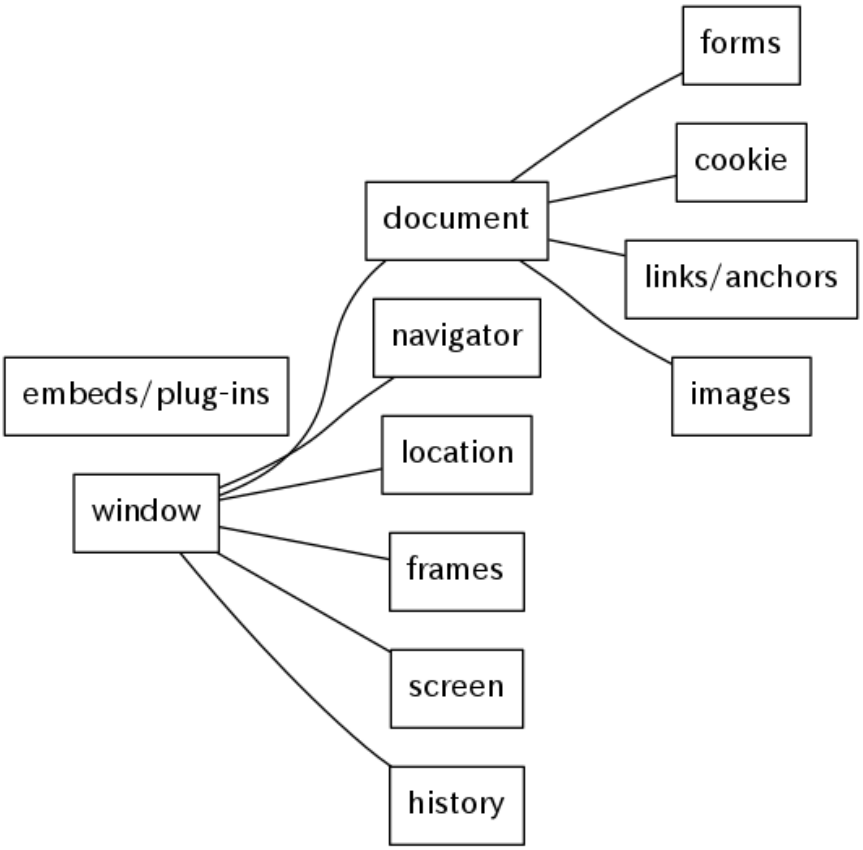
forms
cookie
document
links/anchors
navigator
embeds/plug-ins
images
location
window
frames
screen
history

図 2: BOM の構造

- ウィンドウの作成およびウィンドウの制御
- ウィンドウ内部を分割するタイプのドキュメント (フレームや iframe) の作成
- タイマーの生成と制御
- document、navigator、screen などブラウザウィンドウの内容を操作するためのオブジェクトを指すプロパティ群

4.3.3 ウィンドウの作成と制御

window オブジェクトには新しいウィンドウを作成する 1 つのメソッドと 3 種類のダイアログを作成するそれぞれ 3 つのメソッドとがあります。

メソッド	説明
window.open	新しいウィンドウを生成します
alert	メッセージ (警告) を表示します
confirm	ユーザに OK かキャンセルを確認してもらいます
prompt	ユーザにテキストを入力してもらいます

4.3.3.1 window.open メソッド

window オブジェクトの open メソッドでウィンドウを生成し、その内容、サイズ、表示位置などを制御します。ユーザがブラウザの設定でポップアップウィンドウの表示を禁止していると下のコードを実行しても表示されることはありません。

- 1 番目の引数は作成するウィンドウで表示するドキュメントの URL になります。
- 2 番目の引数はウィンドウの名前です。
- 3 番目の引数はウィンドウの外観や挙動を設定するためのものです。

```
window.open("http://oreilly.co.jp/",
            "test",
            "width=600,height=400,toolbar=no,location=no");
```

3 番目の引数の代表的なオプション設定を示します。すべての環境で利用できるわけではないので検証が必要になります。

オプション	利用目的	デフォルト値
alwaysLowered	親ウィンドウの背面に表示される	no

オプション	利用目的	デフォルト値
alwaysRaised	親ウィンドウの前面に表示される	no
dependent	親ウィンドウが閉じられると連動して閉じられる	no
directories	ブックマークバーまたはリンクバーを表示する	yes
height	コンテンツ表示領域の高さ (ピクセル単位)	最小値は 100 ピクセル
width	コンテンツ表示領域の幅 (ピクセル単位)	最小値は 100 ピクセル
outerHeight	ブラウザウィンドウの高さ (ピクセル単位)	最小値は 100 ピクセル
outerWidth	ブラウザウィンドウの幅 (ピクセル単位)	最小値は 100 ピクセル
top	ブラウザウィンドウの画面上を基準とした表示位置	画面内に表示される
left	ブラウザウィンドウの画面左を基準とした表示位置	画面内に表示される
menubar	メニューバーが表示される	ブラウザによる
toolbar	ツールバーが表示される	ブラウザによる
location	アドレスバーに URL が表示される	ブラウザによる
status	ブラウザウィンドウの下部にステータスバーが表示される	ブラウザによる
resizable	リサイズ可能なウィンドウが生成される	ブラウザによる
scrollbars	スクロールバーが表示される	ブラウザによる
modal	モーダルウィンドウとして生成される	
dialog	ダイアログウィンドウを生成する	
minimizable	最小化ボタンを追加、dialog が yes の場合のみ有効	
titlebar	タイトルバーの表示または非表示	on
close	閉じるボタンの表示または非表示	on

4.3.3.2 alert メソッド

ユーザにメッセージを表示します。

```
alert("本当に終了しますか？");
```

4.3.3.3 confirm メソッド

質問のメッセージと「OK」ボタンと「キャンセル」ボタンで構成されます。

```
var result = cofirm("ポテトも一緒にいかがですか？");
```

4.3.3.4 prompt メソッド

説明用のメッセージとテキスト入力用のフィールドと「OK」ボタンと「キャンセル」ボタンで構成されます。下の例の 1 番目の引数は説明用のメッセージ、2 番目の引数はテキスト入力用のフィールドのデフォルト値です。

```
var response = prompt("お住まいの都道府県を教えてください", "
沖縄");
```

4.3.3.5　ウィンドウの操作

ウィンドウのサイズ、フォーカス、位置などを変更するメソッドを列挙します。

- moveTo(0, 0);
- focus();
- blur();
- resizeBy(50, 50);
- resizeTo(100, 100);

window オブジェクトへの参照はスクリプトを実行中のウィンドウは self、その親ウィンドウは opener です。スクリプト上から作成したウィンドウへの参照が必要な場合は、window.open 呼び出し時の戻り値を取得します。

4.3.4　navigator オブジェクト

navigator オブジェクトは、そのページのアクセスに利用されているブラウザ (あるいはほかのエージェント) に関する情報を提供します。 OS やブラウザ、セキュリティポリシー、言語、クッキーが利用可能かどうかといったものが含まれています。

多くの環境で利用できるプロパティを列挙します。

- appCodeName

  ブラウザのコードネーム

- appName

  ブラウザの名称

- appMivorVersion

  ブラウザのバージョン番号のうちマイナーバージョンの値 (たとえばバージョン 1.52 であれば 52)

- appVersion

  ブラウザのバージョン番号のうちメジャーバージョンの値 (たとえばバージョン 1.52 であれば 1.00)

- cookieEnabled

  クッキーが利用可能がどうか

- mimeTypes

  サポートされる MIME(Multipurpose Internet Mail Extension) タイプが記憶された配列

- onLine

  ユーザがオンライン状態かどうか

- platform

  ブラウザを実行中のプラットフォーム

- plugins

  ブラウザがサポートしているプラグインの情報が記憶された配列

- userAgent

  ブラウザのユーザーエージェント文字列

- userLanguage

  ブラウザがサポートする言語

4.3.5 location オブジェクト

location オブジェクトのプロパティを示します。

プロパティ	利用目的
hash	URL の#以降の部分
host	URL のホスト (ドメイン) 名とポート番号
hostname	URL のホスト (ドメイン) 名
href	完全な URL(代入も可能。代入した URL に移動する)
pathname	URL のパス名の部分
protocol	URL で利用されているプロトコル (http、https など)
search	URL の最初に現れるクエスチョンマーク (?) とそれ以降の部分
target	URL にターゲットが指定されていればその名称

4.3.6 タイマー

タイマーには一度だけ動作するものと一定の間隔をおいて繰り返し実行されるものの2種類が存在します。1回だけ動作するタイマーの設定には`setTimeout`メソッドを利用します。このメソッドの呼び出しには2個の引数が必要になります。ひとつは実行する関数リテラルまたは関数名を指定し、もうひとつにはタイマーが実行するまでの時間をミリ秒単位で指定します。そのほかに設定された引数があれば、タイマー実行時に指定の関数への引数として処理されます。タイマーを設定したときに戻される値がタイマーの識別子となり、これを利用してタイマーをキャンセルすることができます。

```
var timer = setTimeout( func, 5000, "param1", parm2, ..., parmn );

// タイマーのキャンセル
clearTimeout( timer );
```

一定間隔で繰り返し実行されるタイマーが必要な場合にはメソッド`setInterval`を利用します。2個の引数が必要で、関数ルメイと実行間隔を指定します。タイマーの識別子が戻り値になります。

```
var timeout = setInterval( "functionName", 5000 );
```

4.3.7 history オブジェクト

history オブジェクトはブラウザのページ読み込み履歴を保持しており、ブラウザ上の「進む」ボタン、「戻る」ボタンに相当する処理を行うためのプロパティとメソッドが用意されています。

HTML5 からヒストリスタックをプログラムから制御する方法が提供されるようになりました。

- window.history.length

  セッションヒストリのエントリ数

- window.history.state

  オブジェクトの現在の状態

- window.history.go(n)

  指定された数のステップ数だけヒストリを進めたり、戻したりします。指定した値が0の場合、現在のページを読み直します。

- window.history.back()

  1 ステップ分ヒストリを戻します。

- window.history.forward()

  1 ステップ分ヒストリを進めます。

- window.history.pushState(data, title [, url])

  セッションヒストリに引数で指定されたデータを付け加えます。

- window.history.replaceState(data, title [, url])

  セッションヒストリの現在のエントリを更新します。

4.3.8 screen オブジェクト

screen オブジェクトには画面サイズや表示可能数といった情報が記憶されています。ブラウザの種類とバージョンによってサポートされるプロパティには差異がありますが下記に示します。

- availTop(もしくは top)

  ウィンドウを配置できるもっとも上側の位置

- availLeft(もしくは left)

  ウィンドウを配置できるもっとも左側の位置

- availWidth(もしくは width)

  ピクセル単位での画面の幅

- availHeight(もしくは height)

  ピクセル単位での画面の高さ

- colorDepth

  画面の色深度

- pixelDepth

  1 ピクセルあたりに利用されるビット数

4.4 クッキーとその後継技術

4.5 WebStorage API

Web アプリケーションは、データを保存する方法が必要になることがあります。 HTML5 の LocalStorage と SessionStorage API は、使い勝手がよく、簡単にデータを保存する方法を提供します。

技術的には WebStorage API と呼ばれます。ほとんどの人はこれを LocalStorage と呼びます。

LocalStorage には 2 つの特徴があります。

- LocalStorage に保存できるデータ量には制限があります。この制限はブラウザごとに異なりますが、たいていは 5MB 以上 (IE 8 はユーザデータとして 10MB 可能です) 保存できます。
- LocalStorage への読み書きは、シングルスレッドのファイル I/O 操作です。

4.5.1 API

LocalStorage を扱うには `window.localStorage` オブジェクトのメソッドにアクセスします。

- setItem(key, value);
- getItem(key);
- removeItem(key);
- clear();

setItem に渡される値は文字列でなければなりません。配列やオブジェクトのような複合データを格納するするために複合データをシリアライズします。シリアライズには JSON を使います。

```
var
  serialized_data,
  object_data;

// localStorage から key_name の値を取り出し、JavaScript オブジェクトに変換 (パース) する
serialized_data= localStorage.getItem( 'key_name' );
object_data = JSON.parse( serialized_data );
```

```
// JavaScript オブジェクトをシリアライズして localStorage に key_name
の値としてセットする
localStorage.setItem( 'key_name', JSON.stringify( object_data );
```

4.6 DOM(ドキュメントオブジェクトモデル)

4.6.1 DOM の木構造

次の DOM の木構造を構成する要素はすべて Node オブジェクトと呼ばれます。

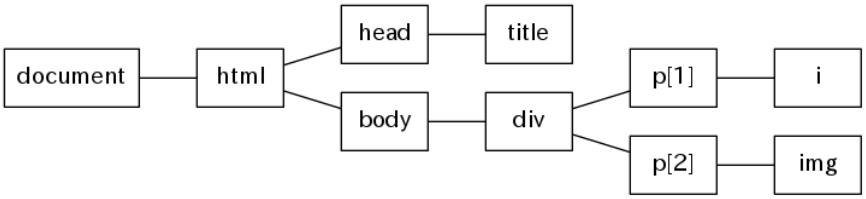

図 3: DOM の構造

4.6.2 Node オブジェクトのプロパティとメソッド

Node オブジェクトのプロパティ:

プロパティ	説明
nodeName	オブジェクトの名称。たとえば、HEAD 要素なら HEAD
nodeValue	要素でない場合は、オブジェクトの値
nodeType	ノードの種類を数値で表現したもの
ParentNode	現在のノードの親ノード
childNodes	子ノードのリスト
firstChild	子ノードリスト中の最初のノード
lastChild	子ノードリスト中の最後のノード
previousSibling	ノードリスト中のひとつ前のノード
nextSibling	ノードリスト中の次のノード
attributes	NaveNodeMap と呼ばれる、要素の属性のキーと値の組からなるリスト
ownerDocument	このノードを保持する document オブジェクト
namespaceURI	ノードの名前空間 URI

プロパティ	説明
prefix	ノードの名前空間プレフィックス
localName	ノードのローカル名 (名前空間 URI が存在する場合)

4.6.3 木構造の変更

document は、すべてのページ要素の親であり、オーナーでもあります。そのため、新しい要素のインスタンスを生成するために利用するファクトリメソッドのほとんどは、Core の document オブジェクトに属します。それに対して、Node オブジェクトは、Core API における木構造の整合性を保つ役割を担当し、親子関係や兄弟関係をたどって、特定のノードに到達することを可能にしています。そして、Element オブジェクトは、入れ子になった要素の中の、限られた範囲にあるものだけを変更するたに、コンテキストを制限する方法を提供します。これらの 3 つのオブジェクトは、いずれもドキュメントツリーの変更には欠かせないものとなっています。

docoment のファクトリメソッド

メソッド	作成されるオブジェクト	説明
createElement(tagname)	Element	tagname に指定された要素を作成
createDocumentFragment()	DocumentFragment	軽量のドキュメント
createTextNode(data)	Text	テキストを保持
createComment(data)	Comment	XML コメント

```
var txtNode = document.createTextNode( "新しいテキストノード" );
```

Node の変更用のメソッド

メソッド	説明
insertBefore(newChild,refChild)	既存のノード (refChild) の前に挿入する
replaceChild(newChild,oldChild)	既存のノード (refChild) と入れ替える
removeChild(oldChild)	既存の子ノードを削除する
appendChild(newChild)	ドキュメントに子ノードを追加する (最後に追加)

4.7 Ajax

Asynchronous JavaScript and XML(Ajax) には次の 2 つの側面があります。

- テクノロジの集合体としての Ajax
- アーキテクチャとしての Ajax

テクノロジの集合体として Ajax は次の構成になります。

- ブラウザとサーバとの間における非同期チャネル
- JavaScript
- XML

アーキテクチャとしての Ajax とは次のようなものです。

- 小さなサーバイベント: Web アプリケーションを構成するコンポーネントは、小さなリクエストをサーバに送り、情報の一部を取得し、DOM に変更を加えることでページの表示を微調整します。ページ全体のリフレッシュは発生しません。
- 非同期: サーバ宛のリクエストによってブラウザがロックすることはありません。ユーザは、アプリケーションの他の部分を使い続けることができます。リクエストの処理が実行中である旨を UI の更新によってユーザにフィードバックすることも可能です。
- onAnything: サーバとの対話は、ユーザによるあらゆるアクションを基に開始できます。マウスのクリックやキーの押下など、今日のブラウザは OS とほとんど同等のユーザイベントを補足できます。あらゆるユーザイベントが非同期リクエストのトリガーとなり得ます。

次のイメージ図のうち、(Ajax) と書かれている部分が Ajax の部分です。

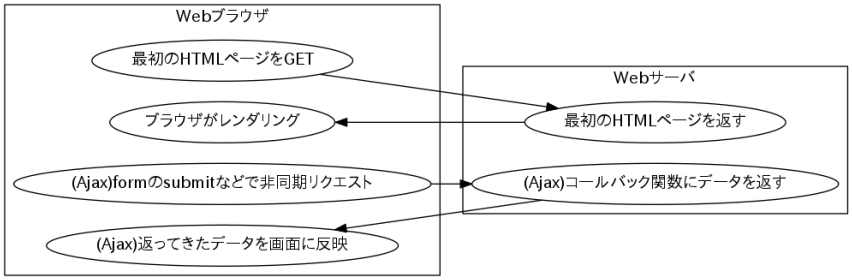

図 4: Ajax のイメージ

4.7.1 XMLHttpRequst

XMLHttpRequst は JavaScript オブジェクトです。これは、HTTP リクエストを送信し、レスポンスを受信し、それを XML として解析できるエンジンです。

リクエストを送信する処理と HTML との結びつけは次のようになります。

1. 例えば type="submit" の input 要素にリクエストを送信する処理をイベントリスナとして登録しておく
2. ユーザが例えば type="text" の input 要素にリクエストデータを入力する
3. ユーザが例えば type="submit" をクリックすることでイベントリスナが呼び出される

Web ブラウザがリクエストを送信する処理は次のようになります。

1. XMLHttpRequst インスタンスを生成
2. onreadyStatechange プロパティに、受信したデータを解析する関数の名前をイベントリスナとしてセットする
3. open() 関数で XMLHttpRequst の設定を行う
4. send() 関数でリクエストを送信する

Web ブラウザがデータを解析する関数は次のようになります。

1. readyState プロパティをチェックして XMLHttpRequst の状態 (進行状況) を確認する。
2. reqdyState プロパティの値が 4(completed) になったら次の処理を行う
3. XMLHttpRequst インスタンスの responseText プロパティからサーバから受信したデータを取得する
4. 受信したデータで DOM を更新する

readyState プロパティが取りうる値は次になります。

readyState プロパティの値	説明
0: (uninitialized)	まだ send() が呼び出されていない
1: (loading)	send() が呼び出され、リクエストの実行中
2: (loaded)	send() が完了して、レスポンス全体を受信した
3: (interactive)	レスポンスの解析中
4: (completed)	レスポンスの解析が終わって利用可能になった

4.7.2 XMLHttpRequst のメソッド

XMLHttpRequst のメソッドを示します。

- open

  open の構文は `open(method, url [, async, username, password])` です。指定した URL への接続を、指定されたメソッド (GET または POST) でオープンします。オプションの引数 async にはリクエストが非同期 (true, デフォルト) か同期 (false) を指定します。またサーバ側処理に必要ならユーザ名 (username) とパスワード (password) を指定します。

- setRequestHeader

  setRequestHeader の構文は `setReuqestHeader(label, value)` です。このメソッドでリクエストヘッダのヘッダ名 (label) と値 (value) のペアを指定します。

- send

  send の構文は `send(content)` です。このメソッドが XMLHttpRequest の核心で、必要なデータとともにリクエストを送信します。

- getAllResponseHeaders

  getAllResponseHeaders の構文は `getAllResponseHeaders()` です。すべてのレスポンスヘッダを文字列として返します。その中にはタイムアウトの値、Keep-Alive、content-type、サーバに関する情報、日付が含まれます。

- getResponseHeader

  getResponseHeader の構文は `getResponseHeader(label)` です。指定したレスポンスヘッダを返します。

- abort abort の構文は `abort()` です。進行中のリクエストを中止します。

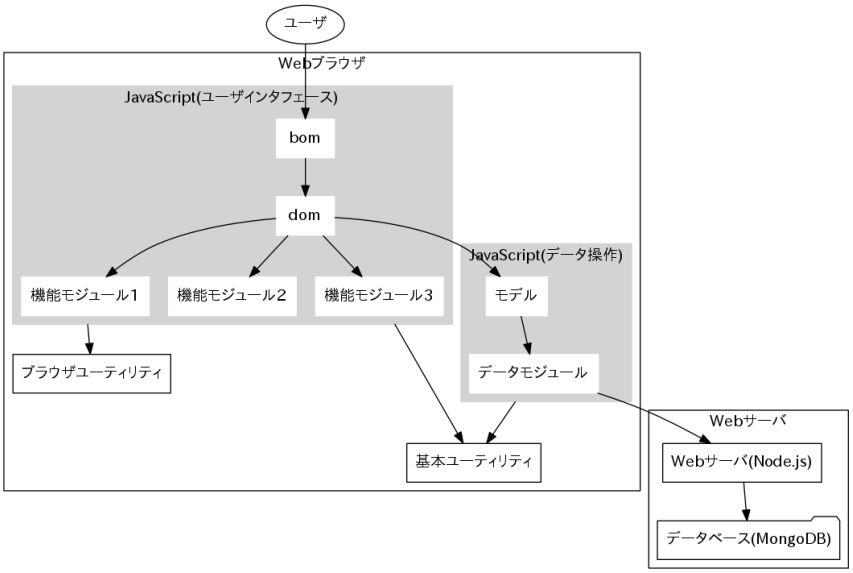

図 5: SPA アーキテクチャ

4.8 デバイス操作 API

4.9 画像とグラフィック

5 SPA クライアント

5.1 シェルの開発

5.2 機能モジュールの追加

5.3 モデルの構築

5.4 モデルとデータモジュールの完成

6 SPA サーバ

Node.js を使用します。

6.1 Web サーバ

6.1.1 サーバの役割

SPA における Web サーバの役割。

- 認証と認可
- 検証
- データの保存と同期

6.1.2 Node.js

Node.js を選択する理由。

- サーバーがアプリケーションである。その結果、別個のアプリケーションサーバーを用意し、インタフェースを取ることを気にする必要がない。すべてを 1 か所で 1 つのプロセスで制御する。
- サーバーアプリケーション言語が JavaScript なので、サーバアプリケーションをある言語で記述し、SPA を別の言語で記述するという言語認識上の負荷を取り除ける。また、クライアントとサーバでコードを共有でき、これには多くの利点がある。例えば、SPA とサーバの両方で同じデータ検証ライブラリを使える。
- Node.js はノンブロッキングでイベント駆動である。一言で言えば、平均的なハードウェアの 1 つの Node.js インスタンスで、リアルタイムメッセージングで使うような大量の並列オープン接続を扱える。多くの場合、これは現代の SPA に大変望ましい機能である。
- Node.js は高速で十分に信頼でき、モジュールや開発者の数が急速に増えてきている。

6.1.3 request オブジェクト

request オブジェクトの重要なプロバティを次に示します。

- ondata: サーバがクライアントからデータを受信するとき (例えば、POST 変数が設定されているとき) に呼び出されるメソッド。これは、ほとんどのフレームワークでクライアントから引数を取得するメソッドとき大幅に異なる。パラメータの完全なリストを変数で入手できるようにこのメソッドを除外する。
- headers: リクエストのヘッダすべて。

- url: 要求されたページの pathname
- method: リクエストに使う方法。 GET または POST。

6.1.4 高度なルーティング

6.1.5 express-session によるセッション管理

Node.js 用のライブラリ、express-session を使います。

expressjs/session

インストール

```
npm install express-session --save
```

使い方

```
var app = express();
app.set('trust proxy', 1);  // trust first proxy
app.use( session({
  secret            : 'keyboard cat',
  resave            : false,
  saveUninitialize  : true,
  cookie            : {secure: true}
}));
```

6.1.5.1 options

次に express-session が受け付けるプロパティを示します。

*は実際に使用しているオプション

- cookie(*)

  session ID cookie を設定します。デフォルト値は`{ path: '/', httpOnly: true, secure: false, maxAge:null }`になります。

- cookie.domain

  Cookie のドメインを指定します。デフォルト値はホストのドメインになります。 URL が要求されているサーバのドメインとの比較に使用します。一致する場合は、cookie.path プロパティを確認します。

- cookie.expires(*)

  永続的な Cookie の有効期限を設定するために使用します。有効期限を 1 時間に設定するときは `new Date( Date.now() + 60 * 60 * 1000 )` とします。

- cookie.httpOnly(*)

  Cookie がクライアントの JavaScript ではなく、HTTP(S) のみで送信されるようにして、クロスサイト・スクリプティング攻撃から保護します。デフォルト値は false です。

- cookie.maxAge

  Cookie の有効期限をミリ秒で設定します。デフォルト値、もしくは null 値が指定されているとブラウザデフォルトの挙動になります。つまり、ブラウザを閉じたら Cookie は削除されます。

- cookie.path

  Cookie のパスを指定します。要求されたパスとの比較に使用します。デフォルト値は'/' になります。このパスがドメインと一致する場合に Cookie を送信します。

- cookie.sameSite

- cookie.secure(*)

  ブラウザーが確実に HTTPS のみを介して Cookie を送信するようにします。 Cookie に署名するかと書かれているサイトもある。デフォルト値は false です。

- genid

- name

- proxy

- resave(*)

  true の場合、セッションストアにアクセスするたびにセッションを作り直します。セッションの作り直しが必要なければ false を指定する。

- rolling

- saveUninitialized(*)

true の場合、未初期化状態のセッションも保存される。特に必要なければ false を指定する。

- secret(*)

  Cookie の暗号化に利用するキーを指定します。キー変更する場合は `secret: ['(新しいキー)', '(今までのキー)', '(今までのキー)']` とします。先頭のキーは暗号化に用いられ、復号化には配列中のいずれかのキーが利用されます。

- store(*)

- unset

6.1.5.2 セッションの破棄

```
req.session.destroy( function ( err ) {
  // cannot access session here
})
```

6.1.6 WebSocket と Socket.io

WebSocket では、ブラウザとサーバが 1 つの TCP 接続上に永続的で軽量な双方向通信チャネルを維持できます。 WebSocket により、ブラウザやサーバは、HTTP リクエスト-レスポンスサイクルのオーバヘッドや遅延なしにリアルタイムにメッセージを送信できます。

Socket.IO は Node.js のモジュールです。 WebSocket が使える場合は WebSocket でブラウザとサーバての間のメッセージングを提供し、ソケットが使えない場合には他の手法を使うように機能を低下させる機能を提供します。

Node.js は優れたメッセージングサーバである。イベントモデルのおかけで、接続ごとにプロセスを作成しない。その代わりに、接続の開閉時に記録を取り、接続中に維持管理を行う。そのため、平均的なハードウェアで何万、何十万もの同時接続に対処できる。 Node.js は、1 つ以上の開かれた接続上でメッセージングイベント (リクエストやレスボンスなど) が発生するまでは何も重要な仕事は行わない。

6.1.6.1 インストール

```
npm install socket.io --save
```

実行結果:

```
└──┬ socket.io@1.7.4
  ├──┬ engine.io@1.8.4
  │  └── ws@1.1.4
  └──┬ socket.io-client@1.7.4
     └──┬ engine.io-client@1.8.4
        └── ws@1.1.2
```

6.1.6.2 Node http server での使用例

Sever(app.js):

```
var
  app = require( 'http' ).createServer( handler ),
  io = require( 'socket.io' )(app),
  fs = require( 'fs' );

app.listen(80);

function handler ( request, response ) {
  fs.readFile( __dirname + '/index.html',
    function ( err, data ) {
      if ( err ) {
        response.writeHead(500);
        return response.end( 'Error loading index.html' );
      }

      response.writeHead(200);
      response.end( data );
    }
  );
}

io.on( 'connection',
  function( socket ) {
    socket.emit( 'news', { hello: 'world' } );
    socket.on( 'my other event',
      function ( data ) {
        console.log( data );
      }
```

```
    );
  }
);
```

Client(index.html):

```
<script src="/socket.io/socket.io.js"></script>
<script>
  var
    socket = io( 'http://localhost' );

  socket.on( 'news',
    function ( data ) {
      console.log( data );
      socket.emit( 'my other event, { my: 'data' } );
    }
  );
</script>
```

6.1.6.3 Experss3/4 での使用例

Sever(app.js):

```
var
  app     = require( 'express' )(),
  server  = require( 'http' ).Server( app ),
  io      = require( 'socket.io' )( server );

app.listen(80);

app.get( '/', function ( request, response ) {
  response.sendfile( __dirname + '/index.html' );
});

io.on( 'connection',
  function( socket ) {
    socket.emit( 'news', { hello: 'world' } );
    socket.on( 'my other event',
      function ( data ) {
        console.log( data );
      }
```

```
    );
  }
);
```

Client(index.html):

```
<script src="/socket.io/socket.io.js"></script>
<script>
  var
    socket = io( 'http://localhost' );

  socket.on( 'news',
    function ( data ) {
      console.log( data );
      socket.emit( 'my other event, { my: 'data' } );
    }
  );
</script>
```

6.1.7 まとめ

6.2 サーバデータベース

MongDB を使用します。

6.2.1 MongDB シェルコマンド

MongDB のシェルは端末を開いて `mongo` コマンドで起動します。データ階層はデータベース -> コレクション -> ドキュメントと理解するといいと思います。

コマンド	説明
show dbs	この MongoDB インスタンスにあるデータベースをすべて一覧表示する
use database_name	現在のデータベースを database_name に切り替える
db	現在のデータベース

コマンド	説明
db.getCollectionNames()	現在のデータベースで利用できるすべてのコレクションのリストを取得する
db.collection_name	現在のコレクション
db.collection_name.insert({'name': 'Josh Powell'})	ドキュメントを挿入する
db.collection_name.find()	ドキュメントをすべて返す
db.collection_name.find({'name': 'Josh Powell'})	name の値が Josh Powell であるすべてのドキュメントを返す
db.collection_name.update({'name': 'Josh Powell'},{'name': 'Mr. Joshua C.Powell'})	name の値が Josh Powell であるすべてのドキュメントをすべて更新する
db.collection_name.update({'name': 'Mr. Joshua C.Powell'},{$set: {'job':'Author'}})	name の値が Mr. Joshua C.Powell であるすべてのドキュメントを部分的に更新する
db.collection_name.remove({'name': 'Mr. Joshua C.Powell'})	name の値が Mr. Joshua C.Powell であるドキュメントをすべて削除する
exit	MongoDB シェルを終了する

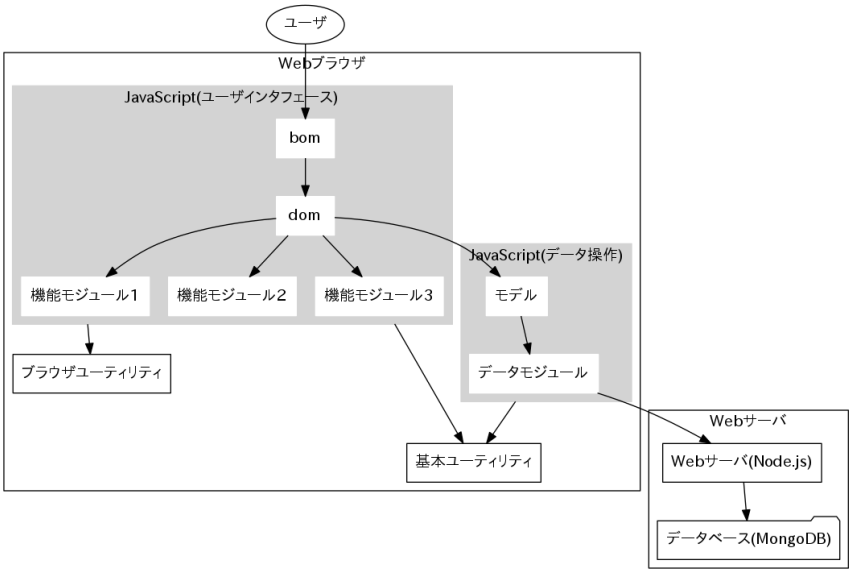

図 6: SPA アーキテクチャ

6.3 SPA を本番環境に備える

7 SPA クライアント

7.1 シェルの開発

7.2 機能モジュールの追加

7.3 モデルの構築

7.4 モデルとデータモジュールの完成

7.4.1 データバインディングと jQuery

データバインディングは、JavaScript のオブジェクトが変更されたときに、その変更を DOM 要素に反映し、逆にユーザがデータを変更するフォームを操作したときに、それに応じて JavaScript のオブジェクトを更新するメカニズムです。

7.4.2 オブザーバ

オブザーバ (Observer) パターンはクライアント側の JavaScript プログラミングでよく使われます。ブラウザイベント (マウスオーバー、キープレスなど) はこの例です。ブラウザが起こしたイベントと比較するために、そのプログラムで作成したイベントをカスタムイベントと言います。このパターンは、購読者/発行者パターンとも言います。

このパターンの狙いは疎結合を促進することです。あるオブジェクトが別なオブジェクトのメソッドを呼び出すかわりに、あるオブジェクトは別なオブジェクトの特別な機能を購読し、そのオブジェクトから通知を受けます。この購読者は観察者とも言われます。一方、観察されているいるオブジェクトは発行者あるいはサブジェクトと呼ばれます。重要なイベントが発生したとき、発行者はすべての購読者に通知 (呼び出し) します。ある形式のイベントオブジェクトでメッセージを渡すこともあります。

8 JavaScript のコーディングルール

8.1 コードレイアウトとコメント

8.2 変数名

8.2.1 命名規則を使ってコメントを減らし改善する

8.2.2 命名規則を使う

8.2.3 関数の命名

関数は大抵オブジェクトに対して処理を行う。そのため、関数名の最初の部分は常に動作動詞にしたい。

- 関数名は、必ず動詞の後に名詞がくるようにする。例えば、get_record や empty_chache_map など。
- モジュールスコープの関数は必ず 2 音節以上にし、スコープを明確にする。例えば、getRecord や emptyCacheMap など。
- 一貫した動詞の意味を用いる。次は一般的な動詞の一貫した意味を示している。

指標	指標の意味	ローカルスコープ	モジュールスコープ
fn(汎用)	汎用的な関数の指標	fn_cync	fnSync
curry	引数で指定された関数を返す	curry_make_user	curryMakeUser
destroy, remove	データ構造を削除する	destory_entry	destroyEntry
empty	データ構造から要素を削除する	empty_chache_map	emptyCacheMap
fetch	外部ソースから取得したデータを返す	fetch_user_list	fetchUserList
get	オブジェクトなどからデータを返す	get_user_list	getUserList
make	新たに生成したオブジェクトを返す	make_user	makeUser
store	ストレージ用の外部ソースにデータを送る	store_user_list	storeUserList
set	引数で指定された値の初期化や更新を行う	set_user_name	setUserName
update	set と同様だが、以前に初期化されている	update_user_name	updateUserName
on	イベントハンドラ	on_mouseover	onMouseover

8.3 変数の宣言と割り当て

8.4 関数

8.5 名前空間

8.6 ファイル名とレイアウト

8.7 構文

8.7.1 ラベル

8.7.2 文

8.7.2.1 continue

8.7.2.2 do

8.7.2.3 for

8.7.2.4 if

if 文は、以下に示す形式のいずれかを使うようにする。 else キーワードは新たな行の先頭から始める。

```
if ( condition ) {
  // 文
}
else if ( condition ) {
  // 文
}
else {
  // 文
}
```

8.7.2.5 return

return 文では、戻り値をかっこで囲むべきではない。セミコロンの挿入を避けるために return キーワードと同じ行で始めなければいけない。

8.7.2.6 switch

8.7.2.7 try

8.7.2.8 while

8.7.2.9 with

8.7.2.10 カンマ演算子を避ける

8.7.2.11 割り当て式を避ける

8.7.2.12 必ず===と!==の比較を使う

8.8 コードを検証する

8.9 モジュール用のテンプレート

9 参考文献

- Front-End Developer Handbook 2017
- Single Page Web Applications Forum
- How to Build a Responsive Form With Flexbox
- MOZILLA DEVELOPER NETWORK
- ブラウザの仕組み: 最新ウェブブラウザの内部構造
- ステートフル JavaScript